Donghyung Lee

DevOps Engineer | Career Description

GitHub: github.com/somaz94 Blog(EN): somaz.blog Blog(KR): somaz.tistory.com Portfolio: somaz.blog/portfolio

Core Competencies

Cloud Infrastructure Design & Operations

Multi/hybrid cloud architecture design and operations experience on AWS and GCP

- **EKS / GKE cost optimization** – 20% via hybrid transition, 50% via GCP migration (Fargate → GKE node pools + MSP discount)
- **Solo greenfield EKS build & launch** – AWS production for a new publishing game — platform 100% GitOps (excluding IAM), keyless authentication via IRSA / Pod Identity
- **IaC standardization** – on-premises, AWS and GCP codified with Terraform and Ansible — 100% IaC for new components (excluding IAM), 100% Git-traceable change history, the same pipeline rebuilt quickly on other projects
- **GPU infrastructure for AI workloads** – GCP A100 / L4 GPU VMs managed in Terraform (NerdyStar), an on-prem GPU box running in-house LLM inference (Phantom)

CI/CD Pipeline & GitOps

Improving development productivity through GitOps-based deployment automation

- **Pipeline design & hardening** – multi-cluster deployment automation on GitHub Actions, GitLab CI, Jenkins and ArgoCD
- **Deploy productivity & reliability** – 67% deployment time reduction, continuous GitOps deployment accelerating time to market
- **Canary zero-downtime rollout** – Blue-Green → Argo Rollouts Canary, removing the 503 race at promotion
- **CI toolchain standardization** – layered shared image plus an automated version cycle, provisioning 5min → ~10s (~97%), 100% tool-version consistency

Monitoring & Observability System Implementation

Establishing proactive incident detection and rapid response systems

- **Centralized logging, 3-stage enhancement** – ELK → EFK → ECK Operator, Elastic Stack 9.x CRD declarative management (automatic TLS rotation)
- **Metrics monitoring** – kube-prometheus-stack alerts and Grafana dashboards, exporters for MySQL, Redis, Elasticsearch and PostgreSQL
- **Distributed tracing** – OpenTelemetry + Tempo completed the Metrics ↔ Traces ↔ Logs 3-signal picture; a split sampling point kept accuracy while cutting cost

Automation & Team Enablement

Operations automation tooling with Python and Bash, plus training and standardized materials that let partner engineers run things themselves

- **Ops toolchain automation** – shell → Python (stdlib only), staged auto-deploy and auto-generated chart-upgrade MRs
- **Engineering enablement** – designed and ran a 6-month hands-on Kubernetes course for the DP (Delivery Partner) field engineers (avg 10 attendees, 4h/session), materials standardized for reuse
- **Personal projects, run in the open** – 10+ public APIs collected, checked and republished twice daily on weekdays (economy dashboard), plus a backend-free diagram editor

Kubernetes Platform Modernization & Open-Source Contribution

Network stack migration, public charts released as open source, and upstream contribution

- **Gateway API migration** – ingress-nginx → a single NGINX Gateway Fabric control-plane, incident-free cutover, wildcard TLS consolidated
- **Open-source shared assets** – Helm charts, Kubernetes CRD controllers, composite GitHub Actions and Ansible collections released on ArtifactHub, GitHub Marketplace and Ansible Galaxy
- **Upstream open-source contribution** – Gateway API HTTPRoute support in apache/airflow, OpenTelemetry and prometheus-community Helm charts; bug fixes and features in kubernetes-sigs (Kueue, KWOK), Prometheus Operator, k3s and Gitea
- **NGINX Gateway Fabric upstream contribution** – policy status conditions and a refactor of 11 complex controller functions, credited as a community contributor in the NGF 2.7 release announcement

Infrastructure Security & IAM

Centralized in-house authentication and secret governance via Keycloak Operator SSO and Vaultwarden

- **Central authentication (SSO/OIDC)** – Keycloak Operator brought single sign-on onto GitLab accounts, with ArgoCD, Harbor and Vaultwarden OIDC integrated
- **In-house secret governance** – Vaultwarden on Kubernetes via Helm, GitLab SSO, automated backups, an interactive restore script
- **GitOps secrets** – plaintext DB passwords → External Secrets Operator + AWS Secrets Manager, pull secret → Sealed Secrets

Professional Experience

Phantom (Concrit Studio)

2024.10 ~ Present

DevOps Engineer

As the sole DevOps infrastructure engineer at a game studio, I design, build and operate a hybrid architecture — dev and qa on-premises, review and prod on AWS — that cut monthly infrastructure cost 20%.
I provide second-line technical support and operations for the live-service game with its external publisher, and build on-premises/AWS infrastructure that raises productivity for new game projects.

Project 1: AWS-On-premises Hybrid Cloud Architecture

Contribution: hybrid architecture design & build 100% (solo) / second-line technical support and operations for the live game in collaboration with the external publisher's ops team 2024.10 ~ Present (In Operation)

Problem

The entire infrastructure ran in a single AWS environment at a high monthly cloud cost.
Non-production environments, whose traffic barely varies, were holding cloud resources around the clock — the estate needed both cost optimization and environment separation.

Approach

- **Hybrid architecture design** – analyzed cost and usage patterns per workload, then moved dev·qa on-premises and kept review·prod, the environments that need scale and high availability, on AWS
- **Network and pipeline isolation** – separated traffic at the application layer rather than standing up a site-to-site VPN, and split the image registry (dev·qa = Harbor, review·prod = ECR) so on-premises and AWS each build and operate against their own registry
- **IaC-based operational standardization** – introduced Terraform and Ansible across both on-premises and AWS, reaching 100% IaC for new components (excluding IAM) and establishing one configuration-management standard across heterogeneous infrastructure
- **IAM credential governance** – only service and CI accounts are managed as code, with console accounts kept separate.
  Terraform never generates user access keys — they are issued and rotated individually — so no long-lived credential is left in plaintext in the Terraform state file

Troubleshooting

- **Version-aware backend routing for game clients** – a single ALB Ingress on EKS had to route old-client and new-client requests to different backends.
  Combined ALB conditional routing rules on a custom HTTP header with target-group mapping, so the split needed no additional infrastructure
- **Stopped L7 connections dropping during rolling deploys** – the ALB deregistration delay and SIGTERM timing were misaligned, so new requests reached terminating Pods and in-flight requests were cut.
  A preStop hook fails the health check first so no new traffic arrives while the rest drains, and RollingUpdate runs add-before-remove to hold capacity — zero connection drops, confirmed on a CloudWatch-backed Grafana dashboard separating ELB and Target 5xx across deploys (zero 5xx, p95 held)

Results

- Cut monthly infrastructure cost 20% by re-placing workloads on the analysis, with the saving reinvested into new project infrastructure
- Physically isolated dev·QA from production, containing the blast radius of any failure within one environment

Project 2: Game Application CI/CD Pipeline & Shared Toolchain Standardization (App GitOps)

Contribution 100% (Sole design and build across the pipeline and CI standardization) 2024.12 ~ Present (In Operation)

Automated the game application's manual build-and-deploy into a GitOps pipeline on GitLab CI and ArgoCD, then standardized the CI toolchain image layers and version governance shared by every in-house repository on top of it, gathering build-environment provisioning and tool-version management into the shared images and one set of central version variables.

2-1. Game Application Deployment CI/CD Pipeline (App GitOps, 2024.12 ~ Present)

Problem

Developers built and deployed to the servers by hand — about 30 minutes per deploy, with human error causing frequent incidents.

Approach

- **GitOps continuous delivery pipeline** – GitLab CI wired to ArgoCD so a Git push drives build → test → deploy end to end, with ArgoCD detecting any mismatch between the configuration declared in Git and the cluster state and self-healing back to it; the deployment logic for

all four environments — dev, qa, review and prod — was unified into that single pipeline, so every environment deploys through the same procedure
- **Least-privilege image build environment** – to avoid granting the host administrator privilege (privileged mode) that Docker-in-Docker (DinD) requires, image builds moved to Kaniko, which builds container images in user space without a Docker daemon — removing the privileged requirement from the build container, which lowers the risk that a compromised build job takes over the host

Troubleshooting

- **Preventing ArgoCD cascading prune** – so a bad Git commit cannot cascade into deleting every child Application, workload Applications reconcile with `prune: true` + `selfHeal: true`, while the app-of-apps root and AppProject layer keep `prune: false` so deletions need manual verification; every sync event is tracked in Slack in real time
- **Removing static SSH keys from the data deployment path** – the CI deploy account held an SSH key and connected straight to the target instances (scp/rsync); that path was replaced by uploading artifacts to an S3 transit bucket and having each instance pull them itself via SSM SendCommand — static SSH keys are eliminated from the deployment environment, closing off the key-leak path

Results

- 67% shorter deployment time (30min → 10min) — the manual build, deploy and configuration steps were unified into the pipeline, eliminating human error as a root cause of deployment incidents
- ~78% shorter source fetch time in the data deployment (~139s → ~30s) — a Git optimization that fetches only the latest commit and only the files actually needed (shallow + blobless fetch) removed the transfer bottleneck
- An average of 100 deploys a week per core service repository — GitOps continuous delivery turns a Git push straight into a deploy, and the time for a new feature to reach production came down
- Built a MasterData pipeline across repositories — a design-data change automatically triggers the deployment pipeline of every downstream repository that consumes it

---

2-2. Shared Toolchain Image Layers & Automated Version Cycle (2025.09 ~ Present)

Problem

Every CI job in the individual repositories re-installed the required tools (helm · helmfile · kubectl · crane · aws-cli) on an Alpine base, so build-environment provisioning averaged 5 minutes.
Tool versions were also fragmented per repository, so adopting a new version meant editing all of them, and the `:latest` tag left it hard to tell which versions were inside an image.

Approach

- **Layer 1 (Mirror)**: mirrored the tool images from the external registries (Docker Hub, GHCR) into the internal Harbor with `crane copy`, multi-arch manifests preserved, removing the external dependency
- **Layer 2 (Custom)**: built five shared images on top of Layer 1 with the tools already installed, so an individual repository has the tools immediately by pinning an image tag
- **Unified version governance**: defined a shared build template and central version variables, so a repository no longer tracks individual tool tags
- **Shared CI templates modularized**: the CI jobs each repository used to copy were consolidated into a shared template repository referenced through `include`.
  The shared anchors and the main jobs (keyless AWS auth, Kaniko build, ArgoCD deploy, backup) are managed in one place, and every repository references the same template, so the common jobs no longer differ from repo to repo
- **Automated version cycle**: everything from upstream version detection to downstream propagation is automated, with the pipeline run and the Tier 2 MR approval left for a person to check:
  · (1) Upstream version detection (weekly scheduler)
  · (2) Layer 1 mirror auto-MR
  · (3) Layer 2 rebuild and central version-variable sync auto-MR
  · (4) Batch auto-propagation to the consumer repos (excluding the template provider repo)

Troubleshooting

- **Toolchain build failure from a cross-tool version conflict**: during an automatic version bump, helmfile 1.5.1's minimum requirement (helm 3.18.6 or later) no longer matched the pinned version (helm 3.16.4), breaking the build pipeline
- **Cause and resolution**: cross-tool dependencies appear only in upstream release notes and cannot be detected in advance, so a minor-guard on helm · helmfile · kubectl auto-bumps only patch versions within the current minor, while minor and major changes are promoted by hand after a compatibility check (with a one-off environment-variable override)

Results

- **CI provisioning time cut ~97% (5min → ~10s)** — the pre-built standard images removed the build delay, with 100% tool-version consistency across the internal repositories
- A 4-tier version governance — the scattered version points, from the build argument through central variable management and consumer indirection to lint-based drift detection, are ordered into a single flow
- A risk-based sequential upgrade pipeline in routine use — per-component risk tiers (T1 weekly / T2 biweekly / T3 monthly + a pre-flight check) drive version detection, Slack alerts and wave-sequenced apply, with major bumps held back and T3 rolled back as a whole on failure

- Automated new-version propagation for helm / helmfile / kubectl / crane / rclone, with the high-blast-radius image rebuild gated behind human review rather than the schedule
- Multi-architecture support — a build path for amd64 · arm64 multi-arch images via `docker buildx`

Project 3: Central Logging & Observability Platform (Collection → Integrity/HA → Product Analytics)

Contribution 100% (Sole design across the full lifecycle — collection · integrity · product analytics) 2024.11 ~ Present

In a log-scattered on-premises environment, advanced centralized logging through three stages, hardened data integrity and high availability (HA) across the log collection layer, then extended the accumulated EFK pipeline into a game user-retention analytics platform.

3-1. Centralized Logging System (ELK → EFK → ECK Operator 3-stage Enhancement, 2024.11 ~ Present)

Problem

Server and container logs were physically scattered across the on-premises environment, so a pod's logs were lost whenever it was recreated; and with no unified observability in place, an incident meant connecting to nodes and pods one by one to find the logs.

Approach

- **A lighter collection and processing tier (ELK → EFK)**: per-node Filebeat collection into a central Logstash tier had grown memory-hungry on the JVM-based Logstash, so collection moved to a lightweight Fluent-bit DaemonSet and parsing to Fluentd, splitting the two roles
- **Declarative cluster management**: Elastic's official Helm chart had stalled for over two years, so the ECK Operator was introduced and lifecycle management moved onto declarative CRDs
- **Extension to the cloud (EKS) with keyless automation**: the on-premises stack was templated for the AWS EKS environment and deployed there, with IRSA applied to index lifecycle management (ILM) and the S3 snapshot backup path so neither needs a static credential

Troubleshooting

- **Recovery blocked by the ECK admission webhook's downgrade refusal**: an automatic on-premises Elastic Stack bump referenced an image listed in the release feed but never published to the registry, causing `ImagePullBackOff`; the immediate rollback was refused by the ECK admission webhook, which blocks downgrades, cutting off recovery and spreading the failure to the operator
- **Cause and improvement**: the core operational risk was defined as the inability to roll back rather than the deployment failure itself. The upgrade pipeline gained a registry existence check for the image, a cluster health check, a snapshot-backup warning on a major version bump, and a rollback procedure that accounts for the webhook constraint, preventing recurrence

Results

- Central aggregation for logs from distributed nodes and ephemeral pods — logs survive a pod being recreated or terminated, and logs once checked by connecting to each pod can be traced in time order in a single Kibana view
- Operations automated and the Elastic Stack major version brought current — the manual work of TLS certificate renewal, account provisioning and node replacement is automated (declaring the version in the CRD is enough for a rolling upgrade), and the Elastic Stack major upgrade stalled for over two years was completed (8.x → 9.x)

3-2. Fluent-bit Log Collector Data Integrity & HA Hardening (2026.03 ~ Present)

Read position preserved, so logs are neither duplicated nor dropped: the SQLite offset DB recording how far each log has been read lives on node-local storage, so the collector remembers its read position across pod restarts — zero duplicate ingestion

Detecting collection that stops without an error: monitoring and alerts detect collection stalling while the process stays up

Logs dropped without a retry: traced in the plugin code why Fluentd discarded failed sends without retrying or alerting during an Elasticsearch rolling restart — reproduced the loss on the production config, then sent each batch as a single request so a failure retries the whole batch, and confirmed every record arrived under the same conditions

---

3-3. Game User Retention Analytics Platform (EFK Logs → Product Analytics, 2026.06 ~ Present)

The EFK pipeline extended into a product-analytics platform: the search-and-incident log infrastructure was extended so an Elasticsearch Transform (every 5 minutes) pivots in-game events to one row per user

Retention metrics on a dashboard: D+1~D+30 retention and cohort analysis were built on the existing EFK pipeline and Kibana alone, with no separate commercial SaaS

Project 4: Developer Productivity Tools (APK Deploy Bot · Doc Automation · LLM Service · Git Mirroring · Static File Server · Self-Service Ops Console)

Contribution 100% (Full system design & development) 2025.02 ~ Present

Designed, developed, and operated 6 internal tools to automate repetitive manual tasks and boost development team productivity, finally consolidating them into a self-service console platform behind a single authentication gate.

4-1. APK Delivery Automation: Slack Bot and Static File Server (2025.02 ~ Present)

Deployed a Python bot on Kubernetes that sends Slack completion messages and download QR codes when a Jenkins APK build finishes — 90% QA delivery-time reduction, version confusion eliminated.
The old file server was unmaintained and lacked a directory UI, search and ZIP batch download, so I built and open-sourced a Go static file server, deployed it with an in-house Helm chart on NFS storage, and moved the build artifacts onto it

4-2. GitLab-Google Drive Document Automation System (2 weeks, 2025.06)

One-way rclone sync of design documents from GitLab into a Google Drive shared drive on push — checksum comparison sends only changed files and only the changed directory is cloned, cutting transfer — the manual Drive upload after every document change went from 10min to 2min (–80%), with GitLab as the single source of truth and direct Drive edits overwritten on the next sync

4-3. Internal LLM Service Deployment (4 weeks, 2025.11)

Built an in-house LLM service in four weeks on an on-prem GPU server with Ollama and Open WebUI on docker compose.
The coding model stays loaded in GPU memory, so requests skip the reload wait, and the service runs today as a shared tool for the dev team

4-4. Git Repository Mirroring Tool + Retry API Follow-up (4-week initial build, 2026.02 ~ Present)

Built and open-sourced git-bridge, a Go mirroring tool, to resolve the gaps and lag of manual syncing between CodeCommit, GitLab and GitHub, automating the sync with failure retry and Slack notifications for real-time status.
A confirmation guard sits in front of data-loss operations such as force-overwrite and delete, the sync-history console's window widened 10× (up to 5,000 events), and a graceful-shutdown drain that finishes in-flight syncs (measured median 2.9s) keeps a deploy from losing a sync. It now runs in house as the standing sync path between the new and old GitLab instances.

4-5. Internal Self-Service Operations Console Platform (2026.07 ~ Present)

Problem

Repetitive operational work such as creating and deleting accounts depended on shell scripts only the infrastructure engineer could run, so every request passed through one person.
The 10 internal account systems were also fragmented, leaving no way to track or audit which accounts and permissions a given person still held.

Approach

- **A single OIDC authentication gateway**: authentication no longer has to be implemented inside each application — a central OIDC gateway in front of the portal admits only authenticated traffic to the backends, and adding a service is a single configuration entry
- **Internal tools and operations services behind one portal**: the existing internal web tools plus 5 new operations services (accounts · Keycloak · Harbor · Elasticsearch · VPN) — 7 applications in all — are served through a single entry point
- **Ten account systems wired in, with an audit trail**: the provisioning APIs of the 10 scattered account systems are wired into the console for unified query and control, and every account change — create, update, delete — is recorded together with who performed it

Troubleshooting

- **Downtime from a race between the CI build and the GitOps sync**: the manifest is committed before the container image has finished building, so ArgoCD synced an image tag the registry did not yet have and the pods fell into `ImagePullBackOff`.
  Six of the eight services use RWO volumes and therefore the `Recreate` strategy, which terminates the existing pod first — so that window showed up as a full outage
- **Cause and improvement**: an ArgoCD PreSync hook now actually pulls the target image tag before the sync phase, so ArgoCD leaves the existing Deployment untouched until the image has landed — removing the full outage caused by a missing image (on the Harbor ops service, a ~180s outage became a ~20s normal pod swap)

Results

- **The operational bottleneck removed through self-service** – designed and built solo over 6 weeks, so the account and permission requests that had to pass through one engineer are now handled by each team directly from the console, within the permissions scoped to that team
- **One entry point and a central governance view** – the 7 internal tools are consolidated behind a single OIDC authentication gate, and the state of the 10 account systems can be queried and audited per person on one screen

Project 5: Kubernetes Cluster & Internal Systems Operations Enhancement

Contribution 100% (Solo design, with the standard procedures captured as automation scripts the team reuses) 2025.12 ~ Present

To keep the cluster running reliably, overhauled monitoring, automated Kubernetes upgrades and Helm management, moved the network controller onto the Gateway API, and went on to resolve storage bottlenecks and modernize the deploy/upgrade pipeline.
Then also took on backup and recovery across the internal systems and the long-stalled major upgrade of the in-house GitLab.

5-1. Monitoring & Alerting Enhancement (2025.12 ~ Present)

Designed custom alerts and Grafana dashboards on kube-prometheus-stack, integrating DB, Node Exporter and Cilium metrics. Alert rules are split by component group and severity, and an inhibit rule silences lower alerts while a higher one fires, so one incident is never notified twice. When Prometheus later went down after its CRD was deleted from outside the cluster and nobody noticed, the cause was a watcher sharing a failure domain with what it watches. A cron watchdog on a bastion outside the cluster now checks every 5 minutes that the monitoring stack is alive (an in-cluster CronJob was rejected for sharing that fate).

5-2. K8s Upgrade Automation & Helm Chart Management Framework (2025.12 ~ Present)

Scripted the Kubespray upgrade procedure and the post-upgrade cluster health checks (nodes, version consistency, etcd, API server, DNS, certificate expiry and more), cutting verification time 90%.
Certificates are caught before they expire by an expiry check and a renewal script, and a Helm framework with canonical-template synchronization standardized chart version management.

5-3. Ingress-nginx → NGINX Gateway Fabric (NGF) Migration (2026.04)

Consolidated the on-premises ingress-nginx instances onto a single NGF control plane with per-class Gateway CRs.
MetalLB IPs were preserved so the cutover needed no DNS or firewall change and completed without incident (30-60s per class), and consolidating onto a wildcard certificate halved the manual renewal work.

5-4. Storage Performance Optimization: etcd / DB SSD Migration and Build I/O Isolation (2 weeks, 2026.04 ~ 2026.05)

Diagnosed GitLab Runner (buildx) disk-I/O spikes during builds slowing HDD-backed etcd and the game DB by whole seconds.
Moved the control plane and DB worker to SSD and fully isolated builds onto a dedicated VM on separate physical hardware; game DB COMMIT latency returned from 9~14s to milliseconds, with zero blocked in-game connections or slowdowns during builds.

5-5. Kubernetes Infrastructure Deployment Automation & Multi-cluster GitOps (ArgoCD) Migration (2026.03 ~ Present)

Problem

The Shell/awk-based infrastructure monorepo could not be test-automated: its YAML manipulation and per-environment branching had grown too complex.
Deployment was a push from CI runners (`helmfile apply`), so drift between the cluster and Git went undetected, and the setup did not scale to multiple clusters.

Approach

- **Tooling refactored onto the Python standard library**: Python3 modules with no pip-installed dependencies, each paired 1:1 with a `unittest` suite, so they run identically on a local machine and on a CI runner
- **Wave-based deployment and upgrade automation**: a six-stage wave order derived from component dependencies, plus a pipeline that detects a new chart version and opens an MR carrying the values diff and the compatibility checks
- **Moved onto an ArgoCD pull-GitOps control plane**: ApplicationSet (Matrix / Git generators) automates deployment across the on-prem and AWS clusters, and existing Helm releases were adopted into ArgoCD management without recreating a single resource and without an interruption
- **GitOps governance and safety rails**: cascading prune blocked at the top layer plus an AppProject whitelist, limiting the blast radius

Results

- Platform releases across the on-prem and AWS clusters are controlled declaratively from a single GitOps control plane
- The legacy shell scripts were migrated wholesale to Python tooling backed by unit tests, making deployment dependable
- Automated chart-upgrade MRs and wave deployment reduced the manual work of tracking new versions and ordering deployments
- Declarative self-healing of the desired state and per-layer prune control closed off the path to a large-scale deletion incident

5-6. Backup & Disaster-Recovery with Backup Verification (2026.08 ~ 2026.09)

Built backup and recovery across 8 internal systems and 2 NAS paths. The verification standard moved from 'the backup job finished' to 'the backup file was written completely and intact', and that surfaced and fixed a defect where a backup job exited 0 even though the database

dump had been truncated.
Left in place, that backup would have failed at restore time.

5-7. Splitting the In-House GitLab in Two, and the Legacy Major Upgrade: a 21-Hop Path (new instance 2024.10 / upgrade 6 days, 2026.08 ~ 2026.09)

Right after joining, built a new bare-metal GitLab for new projects and moved the still-needed repositories across by mirroring. The old containerized GitLab stays for backup and external exposure, synced by git-bridge, my own tool.
The stalled old instance was upgraded along GitLab's official Upgrade Path — 21 hops over 6 days, about 6.5 minutes of downtime on the first hop (including a one-off volume move) — and the nightly DR replica shrank 123G → 39G (–68%).

Project 6: Infrastructure Security & Governance

Contribution 100% (Design, deployment, SSO integration) 2026.04 ~ Present

Brought the account authentication, secret management and remote-access paths that had grown separately per tool under one internal standard.

6-1. Central Authentication (SSO/OIDC) on Keycloak Operator (2026.04 ~ Present)

Consolidated the GitLab OIDC paths — wired separately into each in-house tool (Harbor, ArgoCD, Vaultwarden) — onto a central IdP on the Keycloak Operator.
The permission flow GitLab group → Keycloak group mapper → ArgoCD role now runs through one place, so permissions are managed in one place and a single sign-in covers every tool

6-2. Secret Governance: Vaultwarden · GitOps Secrets (2026.04 ~ Present)

Moved shared internal passwords scattered across mail and messengers onto Vaultwarden behind SSO, one place to manage them.
DB credentials once committed to Git in plaintext now come from AWS Secrets Manager through External Secrets Operator, and the Harbor pull secret is encrypted with Sealed Secrets — closing the path for plaintext credentials into the GitOps repository

6-3. Secure Remote Access to the Internal Network (OpenVPN PKI, 2026.07)

Built an on-premises VPN gateway for the internal network sized for 50 concurrent connections on a PKI, with per-user certificate issuing and revocation automated end to end by idempotent shell scripts

Project 7: AWS EKS Production Game-Launch Platform (New Externally-Published Title)

Contribution 100% (solo design, build & operation) 2026.06 ~ Present (Building & Operating)

Solely built a greenfield EKS production environment for the commercial launch of a new externally-published game (7-1), then hardened it with distributed tracing (7-2), Canary zero-downtime delivery (7-3) and edge security for the game CDN (7-6) ahead of the soft launch (7-4), adding SLI/SLO with error budgets (7-5) once live traffic was there to measure.

7-1. Greenfield EKS Production Infrastructure & Live-traffic Tuning (2026.06 ~ Present)

Problem

The company's existing games were already served from AWS, but under the rule that every product runs in its own AWS account, the new externally-published game had to launch commercially overseas from a new one.
Rather than joining the existing clusters, a highly available EKS production environment was built from scratch in that account.

Approach

- **On-prem + AWS multi-cluster under GitOps**: stood up a greenfield EKS cluster and brought every platform component under one declarative GitOps (ArgoCD) model
- **Compute separated and tuned on Karpenter**: both node pools run on ARM instances — the game services on on-demand so no node is reclaimed mid-session, and CI runners across a diversified ARM multi-family spot pool, cutting cost and minimising reclaim
- **Networking and authentication automated**: AWS Load Balancer Controller, ExternalDNS and a wildcard ACM certificate together turn a declared Ingress into ALB rules, DNS records and HTTPS with no manual step, and every ServiceAccount runs on Pod Identity / IRSA, removing static IAM keys from cluster workloads
- **Logging and secret foundation**: an eck-operator-based HA EFK stack (3 nodes across 3 AZs, EBS gp3, S3 snapshot SLM) plus External Secrets Operator gave the new cluster its own log collection and secret-reference path
- **SSH keys taken out of the deploy path**: client artifacts ship Jenkins → S3 → CloudFront, and server manifests reach the bastion's EFS mount through an S3 transit bucket and SSM SendCommand, so no SSH key is held anywhere in the path

Troubleshooting

- **Node scale-down without dropping game sessions**: do-not-disrupt was ruled out since it only blocks Karpenter's voluntary work (consolidation, AMI refreshes), not a node failure.
  Sessions live in Redis, so pods can move freely: terminating pods finish in-flight requests, every service keeps at least one pod, and nodes are emptied one at a time
- **Idle nodes never reclaimed after a rolling deploy**: diagnosed an over-sized memory request (2 GiB against a measured 101 MiB) as the cause of lopsided node occupancy (92% against 35%) and of consolidation never firing.
  Brought the request down to 512 MiB so nodes filled evenly again, and widened consolidation from empty nodes only to underutilized nodes as well
- **A node replacement packing a service's two pods onto one node**: during an AMI-drift rotation a service's two pods landed on the same replacement node, becoming a single point of failure and leaving that service Degraded for 17 seconds. A node-level spread constraint now bounds the pod-count difference between nodes to one, and a pod that cannot spread waits instead of being squeezed onto an existing node, which Karpenter resolves by provisioning another.
  At the two-replica baseline that alone keeps the pair on separate nodes. It caps the difference between nodes, not the pods per node, so once the HPA takes the count past the number of nodes, several pods do share one.
  The spread is counted per deployment revision, so a canary's new pods are not weighed against the old ones and stall the rollout
- **Consolidation replacing nodes in an endless loop**: putting that same spread constraint on a second service made Karpenter's simulation disagree with the scheduler, deleting and relaunching a node every ~7.5 minutes, 16 times over.
  Traced to that mismatch in underutilization-based consolidation and reverted to empty-node-only, accepting an underused node over repeatedly evicted live pods — cheap at soft-launch volume, with no scale-out yet
- **Traffic reaching a pod that was still starting**: intermittent 5xx errors from the ALB forwarding to a pod mid-startup were resolved with an ALB pod readiness gate — a pod counts as ready only once it has passed the target-group health check

Results

- A production EKS environment built from scratch in the new game's own AWS account, with 100% of the platform manifests under GitOps (excluding IAM)
- Removed the bastion SSH key and the static IAM keys from cluster workloads; the one AWS key left on the CI agent can only assume a role
- Production and review workloads share one NodePool, removing the minimum one node each separate pool would have held
- The GitOps manifests and the CI/CD migration procedure are documented as a runbook, so the work can be handed over and the same environment rebuilt

---

7-2. OpenTelemetry End-to-end Distributed Tracing Pipeline (OTel + Tempo, 2026.06 ~ Present)

Problem

The existing observability stack — EFK for logs, Prometheus for metrics — gave only a limited view of how calls flow between microservices, where the latency bottleneck sits, and how an error propagates across a distributed environment.

Approach

- **Tracing stack and auto-instrumentation**: an OpenTelemetry Collector and Grafana Tempo (S3 backend, wired through EKS Pod Identity with no static key) on AWS EKS, with the OTel Operator's auto-instrumentation injecting the per-language SDK without touching application code (the .NET service is on hold — arm64 nodes are unsupported — and shows only as the caller's span)
- **Metric extraction and storage split at different points**: RED metrics are derived from 100% of spans before sampling, keeping the aggregates exact, while storage goes through tail sampling — every error, every request over 500ms, and 10% of normal requests — to cut storage cost

Results

- **3-signal cross-linking**: metrics, traces and logs are linked so one Grafana view moves between them, making the call flow between services and its bottlenecks visible, and a trace opens just the logs its request wrote, matched by trace_id (the existing logging is Project 3, metrics 5-1)
- **Two-layer routing to close a trace-fragmentation risk**: identified a structural flaw in the multi-replica collector — each app pod's gRPC connection is pinned to a different instance, so a trace that crosses pod boundaries can have its spans scattered, which can break both keep-every-error-trace and the service-graph join
- Split the pipeline into a receiving layer (layer 1) and a sampling layer (layer 2), routing by trace ID through the loadbalancing exporter over a headless Service, so every span of a trace always reaches the same sampling instance
- Validated on live traffic: while connections were pinned, the per-replica metric series stood at 1,650 : 541.
  After trace-ID hashing, the two sampling instances take spans evenly, at 4.09 : 5.18 spans/s
- **Closing the sampling-layer alert gap**: after the split, the existing collector-down alert could not see a full outage of the sampling layer; four alerts were added — sampling layer down, traces dropped undecided on buffer overflow, buffer near full, and policy evaluation errors — to watch for silent trace loss

---

7-3. Zero-downtime Delivery on Argo Rollouts (Blue-Green → Canary, 2026.06 ~ Present)

Problem

Blue-Green promotion swapped the ALB target group wholesale; when the old pods left before the new ones had registered, no pod was left to take traffic, and requests arriving in that gap failed with 503.

Approach

- **Moved to a canary rollout**: with no stepped traffic shifting, pods are replaced inside the same ALB target group with the new pod ready before the old one is withdrawn, so the ready count never drops below the desired number
- **Resting on a stateless design**: the game services are stateless HTTP with state held in JWT, Redis and the database (TypeORM), so a request hitting either old or new mid-rollout gets the same result.
  A change that cannot have both versions live at once — an API contract change, say — does not ship this way and waits for a scheduled maintenance window
- **Entry and exit timing aligned**: an ALB pod readiness gate marks a pod ready only once the ALB registers it healthy, and on exit the deregistration delay (30s) fits inside preStop (35s), inside terminationGracePeriodSeconds (60s), so a pod leaves the ALB before stopping — 75s for the .NET service, which drains its own requests on SIGTERM
- **Strategy templated, controller made HA**: the shared Helm chart switches a service between a Deployment and a Rollout through its values, and canary is applied only to the production game's public-facing main service (the same game's internal services stay ordinary Deployments), while the controller runs two leader-elected replicas behind a PDB so node replacement never takes both down

Results

- Replacing pods without the ready count ever dropping below desired means a production game deploy no longer leaves a moment with no pod to take traffic, or the 503s that came with it; backward-compatible patches ship with no downtime

---

7-4. Soft-Launch Operation (2026.07 ~ Present)

Results

- Soft launch went live, with its small-scale real traffic served on the Canary deploys and 3-signal observability built above
- Observability, deployment and autoscaling (HPA · Karpenter) all in place for the global launch (no scale-out has fired yet at soft-launch volume)
- Re-sized Pod requests and limits from two weeks of real soft-launch traffic, fixing requests so over-sized that the HPA could never scale out and cutting requested CPU by 48% and memory by 71%

7-5. SLI/SLO and Error Budgets (2026.08)

Problem

Live traffic began with the soft launch, but there was no numeric definition of a healthy service.
Whether something was an incident came down to someone's judgement, and there was nothing to rank stability work against.

Approach

- **SLIs only from signals that can be measured**: rather than fixing a target first and fitting the data to it, request latency taken from every span before sampling was checked to compute correctly and only then recorded as an SLI.
  SLO targets are compared by switching them on the dashboard, and committing to one waits until a baseline has built up
- **No availability SLI yet**: the game server answers HTTP 200 even when it throws, so of roughly 7.9M request spans not one was marked as an error.
  An availability SLI built on that would read 100% through an outage and never burn error budget, so it waits until the application returns errors as errors

Results

- Built latency SLI recording rules and an SLI/SLO dashboard that compares error-budget burn across switchable targets and latency thresholds
- Screened target candidates against the measured distribution, dropping ones such as "p95 under 2s" that would always pass and so measure nothing

---

7-6. Edge Security for the Game CDN: AWS WAFv2 (2026.07 ~ 2026.09)

Problem

Ahead of launch, the game's CloudFront distribution faced the internet with no WAF in front of it, so nothing filtered web attacks or known-bad IPs — the largest open security gap before launch.

Approach

- **Observe in Count mode first to avoid false positives**: four AWS managed rule groups were deployed in Count mode, which tallies matches without blocking, to see rule by rule what live traffic trips them.
  A false positive gets an exception for that sub-rule alone, and once review is done a single tfvars line switches the groups to Block — so no legitimate player is blocked by rules nobody checked
- **WCU capacity and cost estimated up front**: the four managed groups fit in 1,125 WCU, inside the 1,500 default ceiling, and cost was estimated at about $9/month at current traffic, rising to about $70/month at full production traffic

- **Rate-based rules held back**: mobile carriers often put many subscribers behind one public IPv4 address with CGNAT, so an IP-keyed rate limit risked blocking legitimate players, and at a peak of 28 requests per 5 minutes there was no data to set a threshold from.
  The documented plan is to size it from the IP distribution once traffic builds, and to consider counting by a session header instead of by IP
- **Internal paths blocked from day one**: while the managed groups only observe, a separate rule already blocks requests to the internal API paths from anywhere outside the operators' IP allow-list

Results

- WAFv2 put in front of the game CDN with no effect on legitimate players; the four managed rule groups are observing live traffic, and the Count-to-Block procedure and the conditions for a rate-based rule are written down

---

Project 8: Second EKS Cluster on Auto Mode & a Site-to-Site VPN (WireGuard) Gateway

Contribution 100% (solo design & build) 2026.08 ~ Present (Building & Migrating)

Problem

The new publishing title had to reach the publisher's internal authentication account over a private route, requiring VPC peering. The production VPC's address range failed the peering requirement, so the VPC, cluster and data tier had to be rebuilt, and the publisher asked for EKS Auto Mode.
The old cluster ran self-operated Karpenter on managed node groups, with many Terraform-managed compute resources and node lifecycle, spot interruption and core addons all hand-managed. VPC peering is also non-transitive, so the on-premises path to the internal ALB was built separately.

Approach

- **Compute layer moved onto EKS Auto Mode**: node provisioning and lifecycle, the AWS resources Karpenter needs (IAM, SQS, EventBridge) and the built-in addons (LB controller, EBS CSI) are left to AWS, and the code keeps only the NodePool declarations
- **A bootstrap order that deploys NodePools before ArgoCD**: turning off the built-in general-purpose pool (fixed to amd64 and on-demand) to run every pool on Graviton (arm64) and the build pool on Spot leaves only the system pool, whose `CriticalAddonsOnly` taint gives an ordinary pod nowhere to run until your own NodePool exists.
  Delivered by ArgoCD, the NodePools would leave ArgoCD itself Pending on a rebuild, so they and the IngressClass stay in a Tier 0 stage applied by helmfile before ArgoCD
- **Redesigned around the IPAM constraint**: four /20s fill the allocated /18 exactly, leaving no room for more subnets, so instead of giving the data tier subnets of its own, isolation moves to security groups inside the per-AZ private subnets — and the code that moves Aurora, Valkey and EFS onto that layout was written ahead of the cutover
- **Site-to-site WireGuard VPN gateway**: another public ALB would give one service a second hostname, certificate and DNS record, multiplying with every new environment, so a tunnel keeps one domain and certificate while internal traffic stays private.
  The tunnel dials out from on-premises to AWS (EIP), with the peer endpoint left empty so the far-side address is learned from the first handshake
- **The VPC static route managed in one place, the office router**: per-node routes would cut off only the pods on a node missing its route, so the same service would succeed or fail by pod placement; on the router, a broken path shows up on every host at once
- **Dependencies outside Terraform, managed on purpose**: the three dependencies `terraform plan` cannot see (the router's static route, the gateway config file and the private key) are documented in the runbook, and the private key is created outside Terraform, so it never sits in state in plaintext, as an AWS SSM Parameter Store SecureString the gateway reads at boot
- **Platform components ported for compatibility**: 16 core platform components — observability, security, the CI toolchain — were reworked for Auto Mode, and 10 of them moved onto the ArgoCD pull-GitOps track

Troubleshooting

- **Auto Mode ignoring the ALB group annotation**: the controller built into Auto Mode ignores the `group.name` annotation that grouped several Ingresses onto one ALB, and groups only through `IngressClassParams`. Caught when an ALB's stack tag carried an Ingress name instead of the group name; left as-is, 4 of the 5 service-group Ingresses and the public host would have had no ALB.
  Each ALB group now has its own IngressClass and IngressClassParams, and since moving an Ingress to a new class recreates its ALB, each Ingress was deleted first to avoid a name collision
- **NodeClass moved API group, and one addon is not included**: on Auto Mode, NodeClass moved from `karpenter.k8s.aws/v1` EC2NodeClass to `eks.amazonaws.com/v1` NodeClass, so the existing manifests no longer applied.
  The specs were rewritten against the new CRD, and the EFS CSI driver — which Auto Mode does not include — is deployed separately

Results

- On the new cluster, Terraform-managed resources in the compute layer drop from 102 to 17, an 83% reduction: AWS runs the Karpenter controller along with its IAM and interruption-event plumbing, and only the node-pool policy is still managed directly, as NodePools
- Built a site-to-site VPN gateway between on-premises and AWS, keeping the internal ALB off the public internet and preserving a single internal hostname and a single TLS certificate however many environments are added

NerdyStar

2023.03 ~ 2024.07

DevOps Engineer

Worked as a DevOps Engineer at a game/blockchain startup, leading the infrastructure side of a large-scale AWS → GCP migration.
Drove the move for cost optimization and GCP's global network, completing the full service migration within a scheduled game-maintenance window with minimal downtime. Kept dual source management (GitLab on-premises, GitHub for production) and upgraded the stalled bare-metal GitLab in place along the official path (about 12 hops).
Also built a pipeline consolidating MongoDB · CloudSQL · GA · Dune data into BigQuery with auto-refreshed Google Sheets, replacing analysts' 2-3 hours of daily manual collection.

Project 1: Large-scale AWS → GCP Cloud Migration & CI/CD Build-out

Contribution: Infra architecture design, migration strategy & execution lead 70% (remaining 30% = server-team application-side migration collaboration), CI/CD pipeline construction 100% 10 months (2023.03 ~ 2023.12)

Problem

AWS costs kept rising, driven largely by per-Pod billing for the always-on game workloads on Fargate.
The migration targeted cost reduction alongside the global load balancing the multi-region game service needed.

Approach

- **Three-phase sequential migration** - Phase 1 built and validated the dev environment on GCP, Phase 2 QA, Phase 3 production
- **Network and authentication** - Shared VPC (Host / Service Project split); GitHub Actions' GCP auth moved to Workload Identity Federation, replacing service-account keys with short-lived OIDC tokens.
  DNS was pre-validated by subdomain delegation (Hosting.kr → AWS Route53 → GCP Cloud DNS), leaving only the final NS swap for an announced maintenance window
- **Compute re-platforming** - AWS Fargate workloads moved onto GKE standard node pools; per-Pod billing loses to per-VM billing for always-on workloads, so GKE Autopilot — the same billing model — was ruled out
- **Storage minimum bypassed, WAF** - worked around Filestore's 2.5TB SSD minimum by self-hosting an NFS server on Compute Engine (pd-balanced 1TB disk), plus Cloud Armor region blocking and an IP allow-list WAF policy
- **IaC and GitOps pipeline** - codified GCP infrastructure with Terraform (100% excluding IAM), then after the migration built a GitLab CI + ArgoCD GitOps CI/CD pipeline standardizing per-environment config with Helm Charts

Troubleshooting

- AWS ALB sends traffic to every target once all of them fail health checks, where the GCP load balancer blocks it — so health-check failures AWS had been hiding turned into dropped traffic the moment the service moved.
  Rebuilt the health checks and backends around GKE attaching pods directly as NEG endpoints
- A 503 came from the GKE Ingress health check: the path set in BackendConfig returned nothing.
  Made that path always answer and aligned it with the ManagedCertificate and FrontendConfig settings, restoring external traffic
- After the move to Cloud CDN, the HTTP → HTTPS redirect was missing and game client downloads failed.
  Added a separate HTTP URL map that only redirects to HTTPS and pointed the port-80 forwarding rule at it

Results

- Cut cloud cost 50% through the AWS → GCP migration, the move from Fargate to GKE standard node pools, and an MSP reseller discount combined
- Completed the full migration in three phases: AWS kept serving while the same setup was built fresh on GCP, and only the final DNS cutover ran inside an announced maintenance window of about two hours, keeping downtime short
- Codified the GCP infrastructure as Terraform IaC, IAM aside, with every deployment traceable through Git commits
- Eliminated GCP service-account key management overhead and key-leak risk for GitHub Actions by adopting Workload Identity Federation

---

Project 2: Game Data Collection Automation System

Contribution 100% (Python script development & operations) 3 months (2024.01 ~ 2024.03)

Problem

Game and blockchain data was collected manually, costing analysts 2-3 hours daily and causing frequent data gaps from human error.

Approach

- **Multi-source ingestion into BigQuery** - MongoDB via a Dataflow Flex Template ETL, CloudSQL via BigQuery's direct connection, Google Analytics · Dune via their own APIs (Dune with a separate API key), all normalized into a common schema
- **Auto-refreshed analyst-facing Google Sheets** - Python Cloud Functions push BigQuery query results into Google Sheets via the Sheets API, triggered by Cloud Scheduler on Daily (previous-day) and Monthly (previous-month) cadences
- **Full infrastructure as Terraform IaC** - BigQuery datasets, Cloud Functions, schedulers, Cloud Storage, Artifact Registry, IAM and service accounts all managed as code

Troubleshooting

- The CloudSQL → BigQuery federated query failed on a region mismatch and missing connection-SA permissions; fixed by creating the connection in the same region and granting that SA the Cloud SQL Client role
- Dune API's async execute → poll → fetch flow and rate limits caused missing results and timeouts; status polling with backoff now guarantees retrieval, and the API key moved into Secret Manager
- Daily re-runs loaded duplicate records into BigQuery. Applied MERGE and ROW_NUMBER()-based reconciliation logic in the dedup Cloud Function to make the load idempotent
- Writing BigQuery results through the Google Sheets API failed on the per-minute quota (429) and on results too large for one write; results now go 1,000 rows at a time into per-date sheets, with pauses between calls and exponential-backoff retries on quota errors

Results

- Replaced analysts' 2-3 hours of daily manual collection with automated loading, ending the gaps from manual-entry mistakes
- With the whole stack in Terraform, the same pipeline was rebuilt quickly in three other GCP projects, every change traceable in Git

---

Project 3: GPU Infrastructure Standardization & Scale-out for AI Workloads (Windows → Linux · Automated Setup · On-prem / GCP / External IDC)

Contribution 100% (Infra design, build & operations) 6 months (2024.02 ~ 2024.07)

Problem

The in-house GPU server ran Windows, so every Stable Diffusion-family image-generation run meant hand-matching driver, CUDA and framework versions — hard to reproduce, and slow to get to work despite the hardware.

Approach

- **Windows → Linux standardization** - GPU servers moved to Ubuntu 22.04; one setup script covers the driver stack (nvidia-driver-550 · CUDA 11.8 · cuDNN 8.6), GPU checks, the shared NFS model store and a Stable Diffusion WebUI per GPU, plus kohya_ss and ComfyUI
- **Cloud scale-out on GCP** - A100 40GB and L4 VMs in Terraform, reshaped with demand up to four A100s and eight L4s, joined to the migration's Shared VPC subnet after zone availability checks and quota increases
- **20 more servers at an external IDC, card tier split by workload** - 20 GPU servers from the same script; training runs on GCP A100s for their VRAM, and bulk inference, which scales with GPU count, on consumer RTX 40-series servers at the IDC

Results

- Replaced Windows hand-configuration with Linux and a setup script, the standard build for every in-house, GCP and IDC GPU server
- GCP GPU VMs managed in Terraform, so every reshaping and rebuild is tracked in code — folded into the existing GCP IaC repository
- Scaled GPU capacity from one in-house box to 10 in-house, GCP A100/L4 VMs and 20 IDC servers, card tiers split by workload

---

Iorchard

2022.04 ~ 2023.03

Infra Engineer

Built PaaS (Kubernetes + OpenStack + Ceph) infrastructure solutions for clients and provided technical support.
Designed and built private cloud infrastructure for financial and public sector clients, and led training for partner engineers.

Project 1: Customized PaaS Solution for Clients

Contribution 100% (Infra design & Kubernetes cluster build) 11 months (2022.04 ~ 2023.03)

Problem

Each client had different environment and server spec requirements, making standardized solutions insufficient.

Approach

- **HA cluster design** - Kubernetes clusters designed for high availability per client requirements, with VM management built on OpenStack
- **Unified storage** - block, file and object storage provided together through Ceph

Troubleshooting

- **Planned shutdown and restart** - client power work meant cutting power, so zero downtime was impossible and the goal became stopping without corrupting data and coming back cleanly. VMs stop first, then the DB and message queue, with storage last, and restart runs in reverse.
  The DB cluster (MariaDB Galera) is shrunk one node at a time and restarted from the node holding the latest data, and storage (Ceph) is set beforehand so a powered-off node is not treated as a failure that starts data recovery. Restart failures the field could not resolve (a message queue node not rejoining the cluster, volume creation failing) were recovered as second-line support

Results

- Built and delivered the HA-configured PaaS solution to 5 clients and ran it stably
- Automated server provisioning and configuration with Ansible — the same playbooks for every client, and shorter deployments

- Handed the shutdown and restart runbook for power work to field engineers, with the fixes for restart failures written into the same runbook

---

Project 2: Kubernetes Operations Training Program for DP

Contribution 100% (Training design & delivery) 6 months (2022.06 ~ 2022.11)

Problem

DP's field engineers lacked Kubernetes and container experience, so running the PaaS on their own after adoption was expected to be hard.

Approach

- **Step-by-step curriculum** – Kubernetes basics through cluster operations and troubleshooting, taught practice-first in lab environments
- **Parallel PaaS-stack training** – OpenStack and Ceph storage operations taught alongside, covering the full PaaS stack

Results

- Monthly 4-hour sessions for six months (about 10 attendees) and an operations guide let DP's field engineers run Kubernetes on their own with fewer inquiries; the materials were reused for later clients